

Supplemental Material: Detail Enhanced Gaussian Splatting for Large-Scale Volumetric Capture

JULIEN PHILIP*, Eyeline Labs, United Kingdom
 LI MA*, Eyeline Labs, United States of America
 PASCAL CLAUSEN*, Eyeline Labs, Switzerland
 WENQI XIAN*, Eyeline Labs, United States of America
 AHMET LEVENT TAŞEL, Eyeline Labs, Canada
 MINGMING HE, Eyeline Labs, United States of America
 XUEMING YU, Eyeline Labs, United States of America
 DAVID M. GEORGE, Eyeline Labs, United States of America
 NING YU, Eyeline Labs, United States of America
 OLIVER PILARSKI, Eyeline Labs, Germany
 PAUL DEBEVEC, Eyeline Labs, United States of America

CCS Concepts: • **Computing methodologies** → **Rasterization; Image-based rendering; Image processing; Machine learning approaches.**

Additional Key Words and Phrases: Gaussian Splatting, Super Resolution

1 4DGS IMPLEMENTATION DETAILS

1.1 Color Space

We hypothesize that when training 3DGS [Kerbl et al. 2023] or NeRFs [Mildenhall et al. 2020] directly in linear space, because of compressed darks and potential high values, the optimization is badly conditioned. We indeed observe a notable decrease in both PSNR and Perceptual metrics when reconstructing scenes using directly linear colors, whether we optimize against linear values or sRGB tone mapped values. This is visible in Tab. 1, where we compare metrics averaged over 12 scenes, all computed in sRGB space. We use the sRGB color space (Eq. 6) to compute all the metrics for these experiments. sRGB is closer to human perception and also the typical color space used to compute LPIPS and SSIM. We observe that storing the GS in Linear space has a drastic effect on convergence and quality, while optimizing in sRGB space leads to better scores which is expected since metrics are computed in this space.

Below we provide the corresponding rendering formulas following what we presented in the main paper.

*These authors made equal technical contributions.

Authors' addresses: Julien Philip, Eyeline Labs, London, United Kingdom, julien.philip@scanlinevfx.com; Li Ma, Eyeline Labs, Los Angeles, United States of America, li.ma@scanlinevfx.com; Pascal Clausen, Eyeline Labs, Geneva, Switzerland, pascal.clausen@scanlinevfx.com; Wenqi Xian, Eyeline Labs, Los Angeles, United States of America, wenqi.xian@scanlinevfx.com; Ahmet Levent Taşel, Eyeline Labs, Vancouver, Canada, ahmet.tasel@scanlinevfx.com; Mingming He, Eyeline Labs, Los Angeles, United States of America, mingming.he@scanlinevfx.com; Xueming Yu, Eyeline Labs, Los Angeles, United States of America, xueming.yu@scanlinevfx.com; David M. George, Eyeline Labs, Los Angeles, United States of America, david.george@scanlinevfx.com; Ning Yu, Eyeline Labs, Los Angeles, United States of America, ning.yu@scanlinevfx.com; Oliver Pilarski, Eyeline Labs, Munich, Germany, oliver.pilarski@scanlinevfx.com; Paul Debevec, Eyeline Labs, Los Angeles, United States of America, debevec@scanlinevfx.com.

$$C_{\text{linear storage}}^{\text{linear optim}} = (\mathcal{R}(c_G) + B^c) \cdot E^c \quad (1)$$

$$C_{\text{sRGB storage}}^{\text{linear optim}} = \left(\mathcal{R} \left(f_{\text{srgb}}^{-1}(c_G) \right) + B^c \right) \cdot E^c \quad (2)$$

$$C_{\text{linear storage}}^{\text{sRGB optim}} = f_{\text{srgb}} \left((\mathcal{R}(c_G) + B^c) \cdot E^c \right) \quad (3)$$

$$C_{\text{sRGB storage}}^{\text{sRGB optim}} = f_{\text{srgb}} \left(\left(\mathcal{R} \left(f_{\text{srgb}}^{-1}(c_G) \right) + B^c \right) \cdot E^c \right) \quad (4)$$

GS Storage		Optimization	PSNR	SSIM	LPIPS
Color Space		Color Space			
Linear	/	Linear	20.95	0.936	0.103
sRGB	/	Linear	<u>25.78</u>	<u>0.954</u>	<u>0.097</u>
Linear	/	sRGB	21.04	0.931	0.102
sRGB	/	sRGB	26.37	0.962	0.092

Table 1. Comparison of GS quality based on different color optimization space and Gaussian storage space comparisons. Storing the Gaussian colors in unbounded sRGB, space and optimizing in that same space leads to the best results.

We use the following function to go from sRGB to Linear space and Linear to sRGB space. These functions are valid and differentiable on $\mathbb{R}^+$:

$$\text{Linear} = \begin{cases} \frac{\text{sRGB}}{12.92} & \text{if } \text{sRGB} \leq 0.04045, \\ \left(\frac{\text{sRGB} + 0.055}{1.055} \right)^{2.4} & \text{otherwise.} \end{cases} \quad (5)$$

$$\text{sRGB} = \begin{cases} 12.92 \times \text{Linear} & \text{if } \text{Linear} \leq 0.0031308, \\ 1.055 \times (\text{Linear}^{1/2.4}) - 0.055 & \text{otherwise.} \end{cases} \quad (6)$$

1.2 Segment Length

To balance reconstruction quality and enables parallel training, we divide the full sequence into smaller segments and model each

segment as a separate gaussian field. We study the impact of the segment length on the reconstruction quality by training on segment of varying length (from 4 to 16 frames), and evaluating on the same 4 frames. We report PSNR, SSIM, and LPIPS metrics in Tab. 2. Longer sequences require more memory to store all the gaussians and are more computationally heavy as we need to evaluate visibility of all gaussians for each training view and time pair. Shorter sequences are faster to reconstruct but are not as consistent and do not reuse gaussians as much. We found that 8 frames strike a good balance between compute time and reuse of the gaussians. Therefore, we train our *Poly4DGS* representation on sub-sequences of 9 frames, with stride 8. This way there is a one frame overlap helping reduce jumps between segments.

Seg. length	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
4	32.42	0.983	0.0251
6	32.64	0.984	0.0243
8	32.74	0.984	0.0245
10	32.64	0.983	0.0259
12	32.15	0.982	0.0272
16	32.30	0.981	0.0302

Table 2. The effect of segment length for *Poly4DGS* reconstruction. The best results are marked in **bold**.

1.3 4DGS Training Details

During training, *Poly4DGS* is initialized as static, isotropic Gaussians centered at each point both spatially and temporally. The spatial scale is set based on the average distance to the three closet points, and the lifespan (λ_i) of each primitive is initially set to be 3 frames. We use the Adam optimizer for 20,000 iterations using a batch size of 16 split on 4xA100 40GB GPUs. We adopt the following learning rates - means: $1e-4$, scales: $5e-3$, quaternions: $1e-3$, opacities: $5e-2$, $sh0$: $2.5e-3$, shN : $1.25e-4$. We adapt the coarse to fine training strategy of DashGaussian [Chen et al. 2025] to our usecase multiplying the resolution by $2^{\frac{1}{4}}$ every 1,000 iterations to reach 4K training resolution and modifying the rate of densification to follow their policy.

2 DETAIL ENHANCEMENT IMPLEMENTATION DETAILS

2.1 Data and Camera Path Generation

After reconstructing both the high-quality (HQ) and low-quality (LQ) 4DGS for our Face Rig, we randomly generate synthetic camera trajectories with moving cameras to render the training data. These trajectories are designed to span a wide range of focal lengths, positions and orientations, resulting in a dataset that is much more diverse than the original 75-camera rig setup. To generate each trajectory, we randomly sample the start and end camera poses, and interpolate intermediate views with cubic Bézier interpolation. Each camera pose is defined by first sampling the camera-to-face-center distance uniformly in logarithmic space within the range of 0.5 to 50 meters. We then randomly sample the azimuth angle uniformly from -110° to 110° and the evaluation angle from -10° to 70° , which defines a position on a sphere centered around the face. Camera

FOVs are set so that the face occupies between 25% and 100% of the image area.

2.2 Training

We fine-tune our modified version of the FLUX model for 100k using an initial learning rate of $4e-5$ and a cosine learning rate decay. We train on 8x80GB A100 GPUs. To add conditioning to our network, we extend the input weight matrix. Contrary to what is commonly done [Zhang et al. 2023], we do not initialize the added conditioning weights to zero. Instead, we found that initializing them to a small percentage (2-5%) of the original weights leads to faster and better convergence. We hypothesize this is because the signal is available to the DiT from the start, and it does not rely on random gradient to obtain the signal, as also discussed in LightIt [Kocsis et al. 2024].

We then adopt a coarse to fine training strategy to train the diffusion model. Inspired by ICLight [Zhang et al. 2025] that first trains at 512×512 then 1024×1024 , we exponentially increase the size of the patches we train on. We start from 384×384 pixels and reach 1024×1024 pixels at 80% of the training. While we did not observe any difference in terms of validation metrics, this strategy allows training the model about 2.5x faster, going from 100 down to 40 hours. We enable training on up to 1024×1024 pixels patch by using an 8bit version of the Adam Optimizer and DeepSpeed-Zero2 with 16bit training weights and activations. While we control the maximum number of pixels in patches, use various aspect ratios sampled in $[0.5; 2.0]$. At early iterations, we use a batch size of 2 per GPU, until the size of the patch reaches 724×724 , i.e. half of 1024×1024 where we switch to 1 due to memory constraints. To achieve this, we implement a batch-level size synchronization mechanism in the dataset loaders, so each samples from the batch has the same size.

As our training data is rendered in 4096×4096 RGBA linear exp, we can apply on the fly data augmentation such as, uniform resizing, cropping, random white-balance, exposure, slight gamma changes, and modifying the black level. We also Gaussian blur the input LQ RGB and Alpha with a randomly sized kernel of 0 to 16 pixels, to simulate further levels of input quality. These augmentations are synchronized for the input Low Quality (LQ) GS rendering and ground truth High Quality (HQ) target. We finally apply an unbounded sRGB tonemapping since the original network was trained in this space. This data augmentation is important to bridge the gap between the Face Rig and Scene Rig captures, since the lighting setup is not exactly the same. It allows training the model to be robust to diverse appearances.

We replicate the alpha channels of the LQ and HQ GS renders so that they have 3 channels before feeding them to the Encoder, so they are treated as RGB images. Upon decoding, we average the 3 output channels corresponding to the alpha prediction to obtain the final channel.

As mentioned in the main document, in our final experiments we train a single model on all seven actors, mostly to reduce the amount of training needed. We use different text prompts for actors belonging to different groups:

- For the soldiers: "A high quality, 4K frame from a movie with MH and ET."

- For the basketball players: "A high quality, 4K frame from a movie with JW, TK and AV."
- For the walking and running couple: "A high quality, 4K frame from a movie with AS and EW".

We experimented with training three models, one model per subgroup of actors, so they work when multiple actors are together, but did not find any difference in visual quality nor validation losses. This approach is thus also valid but would have required more GPU hours for all our experiments.

2.3 Inference

Inference on 4k -3840x2160- images requires 43 GB of memory which fits both on 80 GB A100 and 48 GB A6000 GPUs, without further compression. We use 16 denoising steps throughout, taking about 30 seconds per frame. We only condition on the previous warped frame and warping mask for the first 8 denoising steps, leaving these channels empty for the last 8 denoising steps. We found that using these conditions on all 16 steps led to slight blur and loss of details that accumulates frame after frame. This is most probably due to VAE Encoding-Decoding. Indeed, each frame is decoded, warped and then re-encoded. We observed in a simple test that after 10 or so encoding-decoding of a static image, strong degradation appears. We also experimented doing the warping in latent space, but nearest neighbor interpolation was too coarse to provide good alignment, and bilinear interpolation led to ghosting artifacts. Allowing the last 8 steps to be purely generative allows reintroducing details while the structure has been fixed in early steps.

We maintain the dynamic range and precision of the linear, exr, input LQ GS renderings during detail enhancement. We first auto-expose the input renderings so 99% of the pixels are in $[0;1]$ before applying an unbounded sRGB tonemapping and feeding them through the VAE and model. We then invert these operations on the predicted output to recover a Detail Enhanced linear exr image.

2.4 VAE Low frequency issues

When experimenting with the Flux [Labs 2024] Model conditioned only on the input Low quality GS, we noticed two types of temporal artifacts. First, the newly created details are not coherent from frame to frame, due to the generative nature of our model. This is mitigated by our reprojection scheme. We also noticed that low frequency flicker was present even with the reprojection that should have helped. We found that the VAE of the model is not able to maintain the brightness level and low frequencies in the image correctly. As can be seen in Fig.1, encoding and decoding a frame through the VAE does not reproduce the input. While some noise is expected due to the compression, it should be local and in the high frequencies. Here we can observe a global shift and low frequencies are modified. While the difference is subtle it is visible in videos especially since this shift does not seem consistent between frames. We stabilize this component by swapping the low frequencies of the output by the one of the input, using pyramids of laplacians as explained in the main paper. For the alpha channel, we didn't need to do that as the network seems able to saturate the output to 0 and 1, which allows to keep clean boundaries.

3 COMPUTE OVERHEAD ANALYSIS

We use NVIDIA A100 GPUs for both the GS reconstructions and enhancement model training. Reconstructing a 200 frames shot using *Poly4DGS*, takes approximately 100 GPU hours. For each actor, generating and rendering both LQ and HQ GS from the *Face Rig* takes about 200 GPU hours, while training the detail enhancement model for all the actors requires an additional 320 GPU hours. Since we typically aim to exploit a dozen shots per capture, the overhead of detail enhancement is less than 50%, without even accounting for compositing and capture time. Thanks to our pipeline, we are able to render impossible shots with complex camera trajectories, going through or between actors; for such complex shots, the benefits outweigh the compute overhead.

4 MORE COMPARISON OF 3DGS AND 4DGS

We evaluate the temporal stability of *Poly4DGS* using t-LPIPS on 20 novel camera trajectories and compare it against per-frame 3DGS reconstruction. *Poly4DGS* achieves a lower t-LPIPS score (0.138) compared to per-frame 3DGS (0.145), indicating improved temporal consistency. We also provide a qualitative comparison in the supplemental video. Per-frame 3DGS exhibits frame-to-frame artifacts, particularly in scenes with significant occlusion between actors, resulting in unstable reconstructions. In contrast, *Poly4DGS* maintains consistent geometry and appearance over time, with clear foreground-background separation across the entire sequence.

5 COMPARISON OF FACE RIG AND SCENE RIG

In Fig.2, we provide comparisons of the *Scene Rig* and *Face Rig* reconstructions from similar points of view, along with a detail-enhanced output from *Scene Rig*. The detail enhancement model restores fine structures, such as hair strands, lip contours, and eyelashes, with noticeable improvement, achieving visual quality comparable to the high-resolution reconstruction in *Face Rig*.

6 ETHICS

In this work, we captured personal data of professional actors such as performance, face shape, and details through our rigs. All actors signed a consent form ahead of the capture. They agreed for the collected data to be used in the context of this rendering and detail enhancement project, including for training a generative model. Our detail enhancement model is designed to enhance and increase the flexibility of existing recorded performance. New performances are not generated. It is meant to be used on the specific actors it was trained on, limiting the risk of it being used for unethical purposes on unwilling subjects. We actively tried to cover a diverse set of actors with different genders and ethnicities. Actors were paid for their performance.

Considering the environmental impact of machine learning, we acknowledge the substantial carbon footprint, water usage, and e-waste associated with AI development and deployment, advocating for more sustainable computational practices.

Beyond environmental concerns, we recognize our social responsibility regarding AI's potential for job displacement and exacerbating inequalities, highlighting the need for proactive strategies for workforce adaptation.



Fig. 1. Left: an input frame. Middle: the resulting frame after encoding and decoding it with the VAE. Right: the difference between the frames, multiplied by 10 and centered around 0.5, for better visibility. We observe that the difference contains low frequencies, as if the decoded output had a slightly different exposure.



Fig. 2. Left: HQ GS reconstruction in *Face Rig*. Middle: GS reconstruction in *Scene Rig*. Right: Final results after detail enhanced.

7 ADDITIONAL RESULTS AND COMPOSITING DETAILS

We present additional qualitative results of our detail enhancement module in Fig. 3. Video results are visible in the supplemental video. As we can observe in this video, we composited the detailed enhanced renderings on virtual backgrounds. The backgrounds were captured with a 14mm fisheye camera and reconstructed through static 3DGS [Kerbl et al. 2023], stylized by video generative process [Runway AI, Inc. 2024]. Compositing foreground and background frames (or plates) requires several manual steps, conducted by highly skilled artists and engineers, to make the foreground and background fit seamlessly together. For the results in this paper we use a simplified compositing workflow in Foundry Nuke, with steps to:

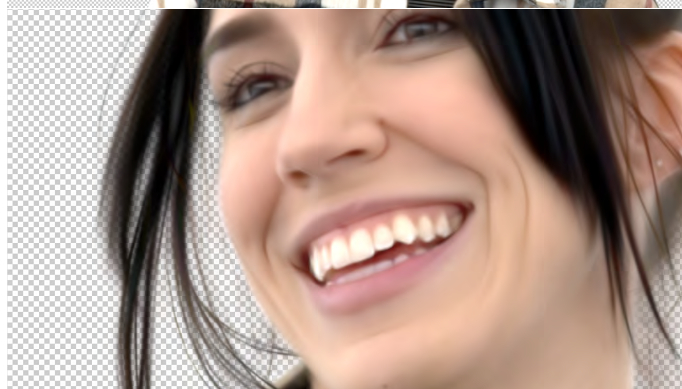
- Integrate color and brightness between elements
- Add optical flow based motion blur
- Blur/defocus the background using a rendered or predicted depth image
- Add contact shadows when ground interactions are in frame
- Refine the alpha channels slightly with light wrapping when the background may motivate spilled light over the edges
- Remove very large floating gaussians using automatic roto-scoping

These steps are part of usual compositing workflows in VFX productions, and we do not claim any contribution to that side. We deliberately avoid using common compositing fixes to ensure our results are presented as accurately as possible while maintaining an integrated look. We also show our results without any compositing for better comparison.

REFERENCES

- Yoyu Chen, Junjun Jiang, Kui Jiang, Xiao Tang, Zhihao Li, Xianming Liu, and Yinyu Nie. 2025. DashGaussian: Optimizing 3D Gaussian Splatting in 200 Seconds. In *CVPR*.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics* 42, 4 (July 2023). <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>
- Peter Kocsis, Julien Philip, Kalyan Sunkavalli, Matthias Nießner, and Yannick Hold-Geoffroy. 2024. LightIt: Illumination Modeling and Control for Diffusion Models. In *CVPR*.
- Black Forest Labs. 2024. *FLUX.1: An advanced state-of-the-art generative deep learning model*. Technical Report. Black Forest Labs. <https://flux1.io/>
- Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. 2020. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. In *ECCV*.
- Runway AI, Inc. 2024. *Gen-3 Alpha*. <https://runwayml.com/research/introducing-gen-3-alpha>
- Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. 2023. Adding conditional control to text-to-image diffusion models. In *ICCV*. 3836–3847.
- Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. 2025. Scaling in-the-wild training for diffusion-based illumination harmonization and editing by imposing consistent light transport. In *The Thirteenth International Conference on Learning Representations*.

Our 4DGS reconstruction



Our Detail Enhanced Results

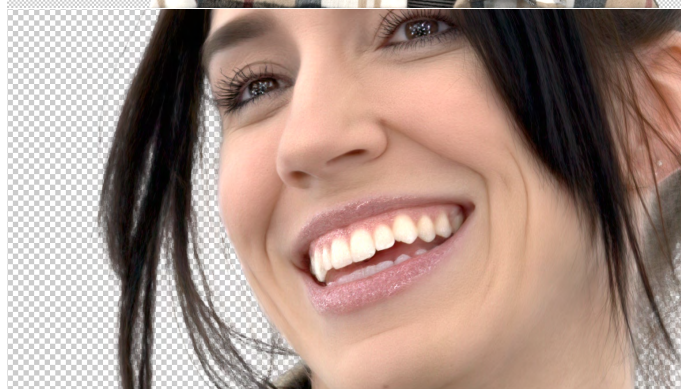




Fig. 3. Left: our 4DGS reconstruction with insets. Right: final results using our detail enhancement module. We can observe that the Gaussian artifacts are effectively removed and new details added.